Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Fundamentos de Arquitectura de Computadores

Bitácora

Marcelo Matey Rodríguez
carné 2025087174

Jorge Pablo Porras Alvarado
carné 2025078613

Josue Adrián Pérez Pérez
carné 2025078641

Profesor: Luis Alberto Chavarría Zamora

4 de noviembre de 2026

1 Bitácora

Fecha	Actividad	Descripción
13/10/2026 al 18/10/2026	Implementación de la `alu_nbit`	Se definieron los módulos de los shifters, multiplicación, suma y resta.
18/10/2026	Investigación del PWM	Se investigó el funcionamiento de un PWM, sus usos y cómo implementarlo; además, se realizó la implementación.
18/10/2026	Idea inicial para los registros de lectura y guardado de contraseñas y entrada	Se implementó un seleccionador para el guardado de los registros. Esto se hizo con base en la idea del proyecto uno, donde se necesitaba una contraseña de abierto y otra de cerrado, y por último el registro para validar. Se reutilizaron los módulos de flip-flop, serializador y deserializador de la tarea 1.
19/10/2026	Planificación del nuevo encoder de 6 bits a 3 bits con una capa de protección para casos fuera de los previstos	Se diseñaron las tablas de verdad y los mapas K.
20/10/2026	Planificación del control del motor y un timeout	Se diseñaron la MEF del control del motor y su diagrama.
20/10/2026	Implementación del encoder	Se hizo la implementación en SystemVerilog del encoder.
22/10/2026 al 01/11/2026	Implementación del control del motor y timeout	Se implementaron en SystemVerilog la MEF de control y el timeout; además, se corrigieron errores del diseño original agregando un estado de error y se arregló la sintaxis.

Fecha	Actividad	Descripción
01/11/2026	Reunión de pruebas y relectura del enunciado	Se hizo una reunión para probar con un top level el guardado de las contraseñas y la lectura de la entrada a validar, y la prueba de los hex y el encoder. Sin embargo, la implementación del `select_storage` tuvo complicaciones, por lo que se decidió releer el enunciado para buscar alternativas y corregir errores de implementación.
02/11/2026	Junta para planificación del top y alternativas	Se hizo una reunión desde la mañana para planificar la solución definitiva para las contraseñas y la creación de una MEF que funcione como manager del top level.
02/11/2026	Día de armado final del circuito y pruebas	Se hizo una reunión para armar el circuito físico y realizar pruebas.

2 Anexos

2.1 Pruebas de trabajo

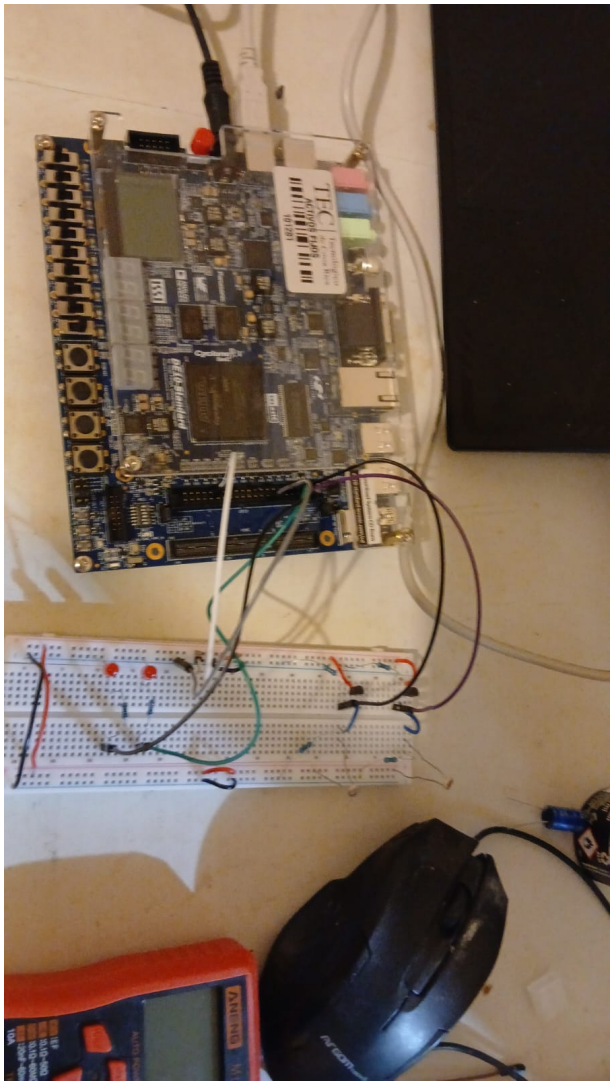


Figura 1: Pruebas del control del motor y el *timeout*.

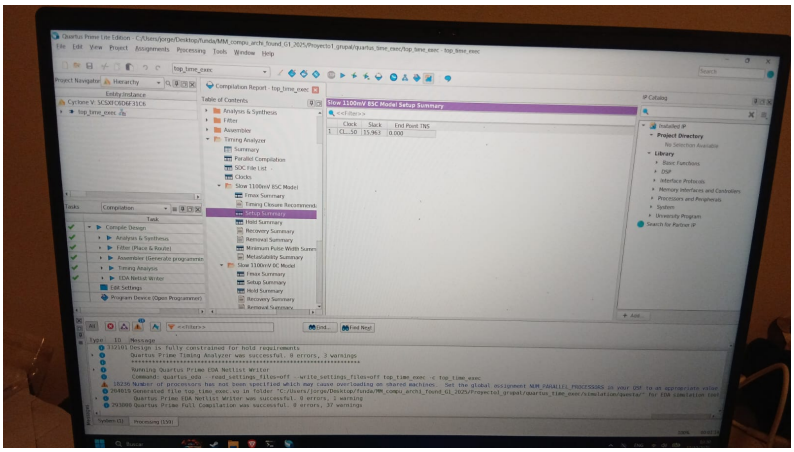


Figura 2: Análisis de temporización: la MEF de *timeout* junto con el *tick* cumple el período de 20 ns con un *slack* alto.

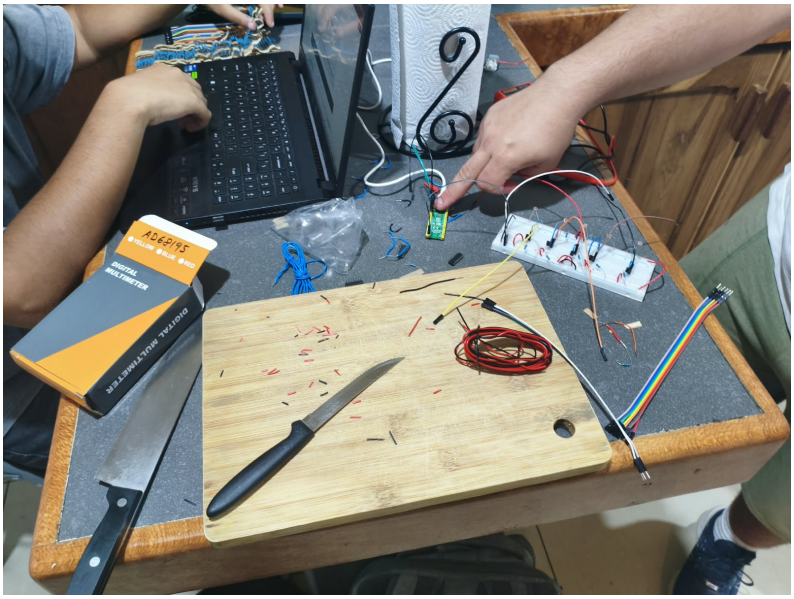


Figura 3: Primer día de pruebas.

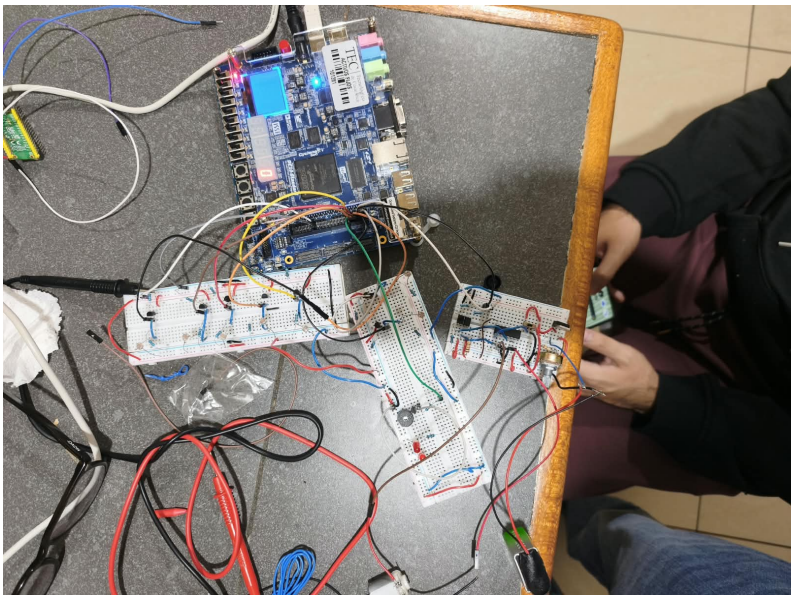


Figura 4: Día final de armado y pruebas.

2.2 Herramientas utilizadas y planteamientos de diseño

2.2.1 Herramientas

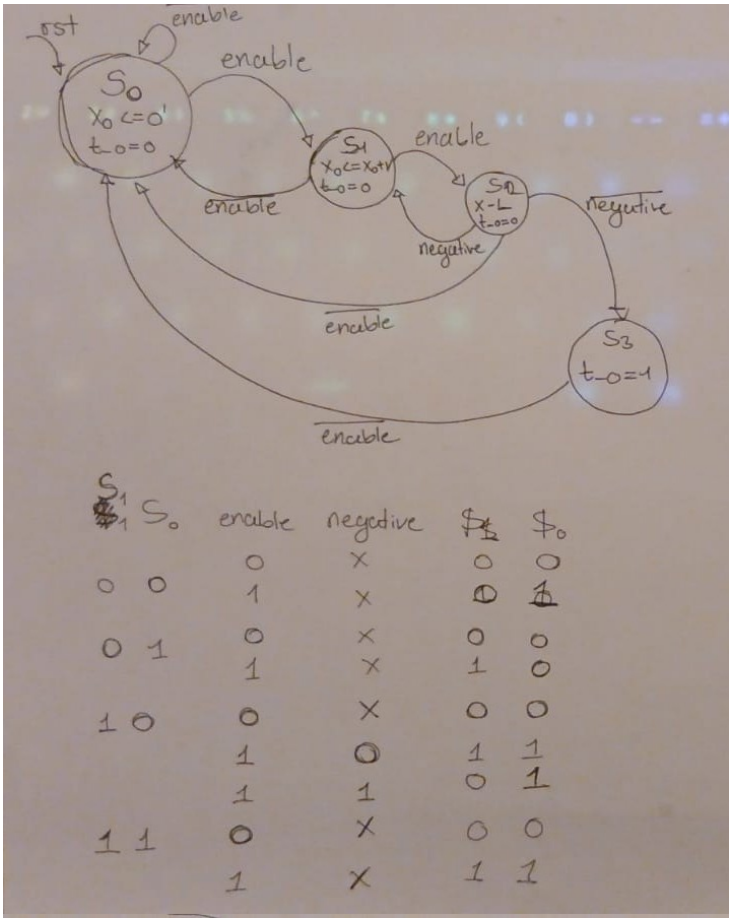
A continuación se enlistan las herramientas utilizadas en la implementación de este proyecto:

- **Software para simulación**: Questa.
- **Software de diseño integrado**: Quartus.
- **Lenguaje HDL**: SystemVerilog.
- **Diagramas (diagrama de la FPGA)**: draw.io.

2.2.2 Diseño

A continuación, se mostrará todo el trabajo de tablas de verdad y mapas K hecho a mano y por medio de LaTeX. Antes que nada, cabe aclarar que los módulos de la `alu_nbit` y los contadores se basaron principalmente en el libro del curso.

2.2.2.1 Control del motor y timeout



S_1	S_0	enable	negative	$\$_1$	$\$_0$
0	0	0	X	0	0
		1	X	0	1
0	1	0	X	0	0
		1	X	1	0
1	0	0	X	0	0
		1	0	1	1
		1	1	0	1
1	1	0	X	0	0
		1	X	1	1

Figura 5: MEF de control del motor y su tabla de verdad.

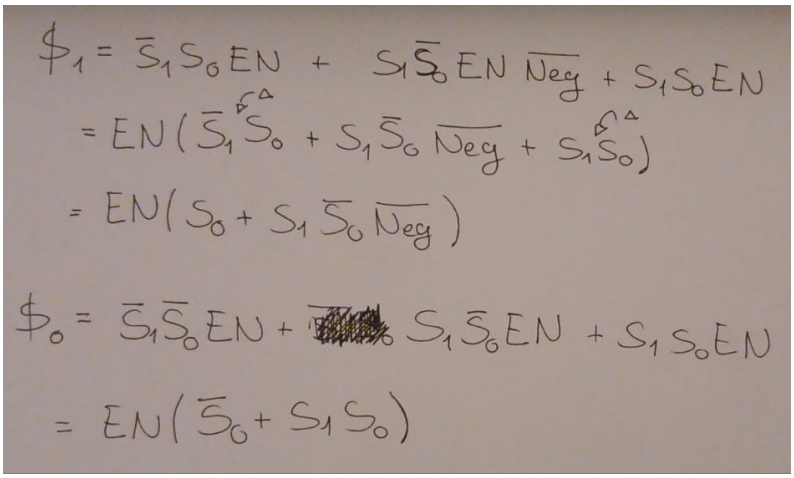


Figura 6: Ecuaciones booleanas para las transiciones de la MEF de control del motor.

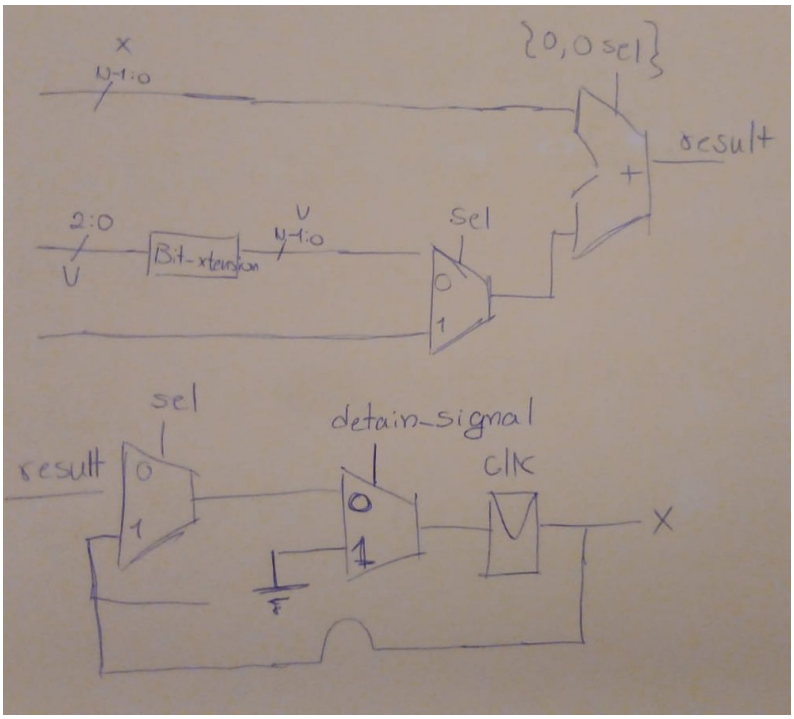


Figura 7: Diagrama del diseño del *timeout*.

2.2.2.2 Encoder

f_5	f_4	f_3	f_2	f_1	f_0	b_0	b_1	b_2
0	0	0	0	0	1	0	0	0
0	0	0	0	1	1	1	0	0
0	0	0	1	1	1	0	1	0
0	0	1	1	1	1	1	1	0
0	1	1	1	1	1	0	0	1
1	1	1	1	1	1	1	0	1

Figura 8: Tabla de verdad del encoder.

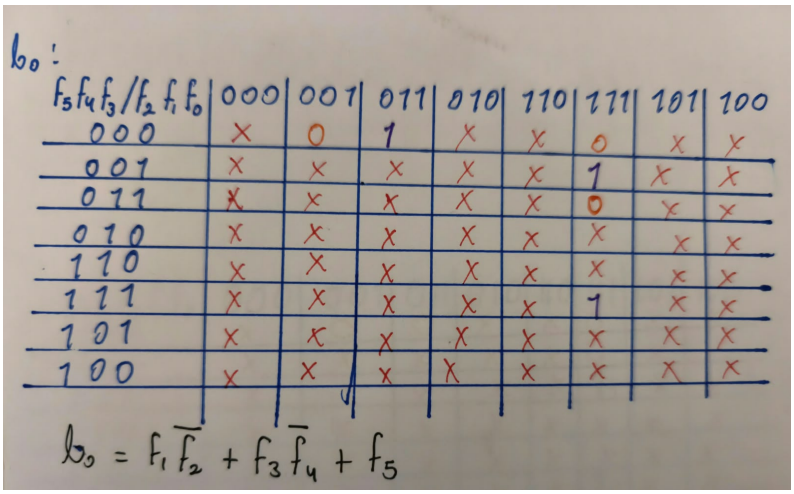


Figura 9: Mapa de Karnaugh de b_0 y su resultado.

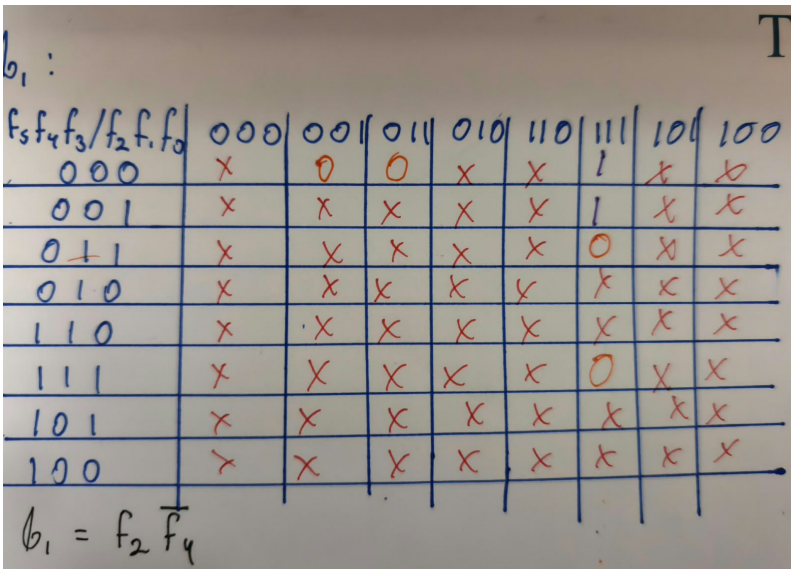


Figura 10: Mapa de Karnaugh de b_1 y su resultado.

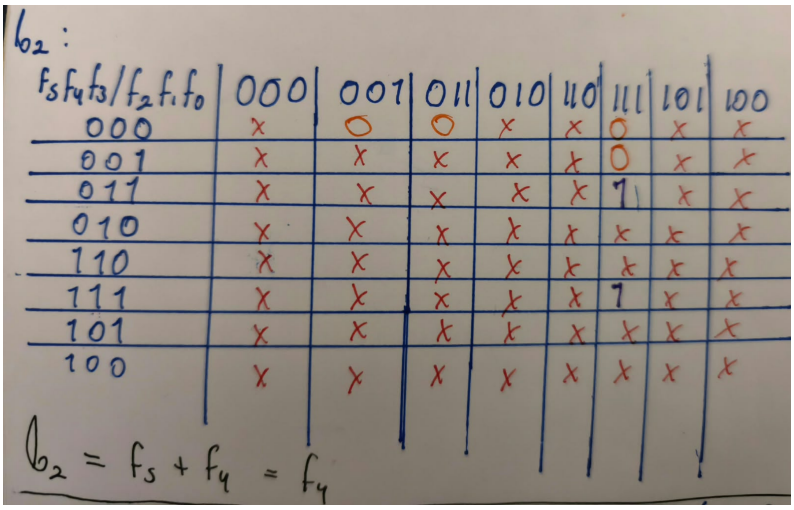


Figura 11: Mapa de Karnaugh de b_2 y su resultado.

2.2.2.3 Validación de la entrada

Para validar que la entrada ingresada es correcta, se verifica que no exista un hueco entre los dígitos de entrada. Sea f_i la bandera asociada a la posición i, con $i \in \{1, 2, 3, 4, 5\}$. Se define el *hueco* en la posición i como

$$\text{gap}_i = f_{i+1} \cdot \overline{f_i}, \qquad i = 1, 2, 3, 4, \tag{1}$$

es decir, $\text{gap}_i = 1$ únicamente cuando $f_i = 0$ y $f_{i+1} = 1$: hay un dígito presente después de una posición vacía.

La entrada es válida si y solo si no existe ningún hueco entre f_1 y f_5, lo cual se obtiene con la compuerta NOR de todos los huecos:

$$\text{valid} = \overline{\text{gap}_1 + \text{gap}_2 + \text{gap}_3 + \text{gap}_4} = \overline{\sum_{i=1}^{4} \text{gap}_i}. \tag{2}$$

Aplicando el teorema de De Morgan y sustituyendo (1):

$$\text{valid} = \prod_{i=1}^{4} \overline{\text{gap}_i} = \prod_{i=1}^{4} \overline{f_{i+1} \cdot \overline{f_i}} = \prod_{i=1}^{4} \left(\overline{f_{i+1}} + f_i\right). \tag{3}$$

Cada factor $\overline{f_{i+1}} + f_i$ equivale a la implicación $f_{i+1} \Rightarrow f_i$. Por lo tanto, valid $= 1$ si y solo si cada bandera activa implica que la anterior también lo está, de modo que los unos forman un bloque contiguo que empieza en f_1 y no hay huecos.

2.2.2.4 password_memory

El módulo guarda hasta cuatro contraseñas de 9 bits. Cada ranura k tiene un bit de válido v_k que indica si está ocupada. Con una sola señal `toggle` se agrega el código si no está registrado y se borra si ya lo está.

2.2.2.4.1 Primera ranura libre

La Tabla 2 indica qué ranura se escribiría según los bits de válido. Solo un w_k vale 1 a la vez, y se activa en la ranura libre de menor índice. Si las cuatro están ocupadas, `full` vale 1.

v_0	v_1	v_2	v_3	w_0	w_1	w_2	w_3	full
0	X	X	X	1	0	0	0	0
1	0	X	X	0	1	0	0	0
1	1	0	X	0	0	1	0	0
1	1	1	0	0	0	0	1	0
1	1	1	1	0	0	0	0	1

Cuadro 2: Primera ranura libre y bandera de memoria llena.

De la Tabla 2 se obtienen:

$$\begin{aligned} w_0 &= \overline{v_0}, & w_1 &= v_0\,\overline{v_1}, \\ w_2 &= v_0\,v_1\,\overline{v_2}, & w_3 &= v_0\,v_1\,v_2\,\overline{v_3}, \\ \text{full} &= v_0\,v_1\,v_2\,v_3. \end{aligned} \tag{4}$$

2.2.2.4.2 Comparación del código con cada ranura

Para saber si `code` ya está registrado, se compara bit a bit con cada ranura usando XNOR (Tabla 3). La salida vale 1 solo si los dos bits son iguales.

code_i	$\text{slot}_{k,i}$	$\text{igual}_{k,i}$
0	0	1
0	1	0
1	0	0
1	1	1

Cuadro 3: XNOR de un bit.

Los nueve bits coinciden si y solo si la AND de todos los XNOR vale 1, y la coincidencia solo cuenta si la ranura es válida (Tabla 4).

match_k	v_k	hit_k
0	0	0
0	1	0
1	0	0
1	1	1

Cuadro 4: Coincidencia válida en la ranura k.

$$\begin{aligned} \text{igual}_{k,i} &= \overline{\text{code}_i \oplus \text{slot}_{k,i}}, \\ \text{match}_k &= \prod_{i=0}^{8} \text{igual}_{k,i}, \\ \text{hit}_k &= \text{match}_k \cdot v_k, \\ \text{hit} &= \text{hit}_0 + \text{hit}_1 + \text{hit}_2 + \text{hit}_3. \end{aligned} \tag{5}$$

2.2.2.4.3 Decisión de agregar o borrar

Con el pulso `toggle` y la señal `hit` se decide la acción (Tabla 5). Si la memoria está llena y el código es nuevo, $w = 0000$ y no se escribe nada.

toggle	hit	full	Acción
0	X	X	Se conserva todo
1	1	X	Se borra la ranura que coincide
1	0	0	Se agrega en la primera ranura libre
1	0	1	No pasa nada (memoria llena)

Cuadro 5: Acción según `toggle`, `hit` y `full`.

$$\begin{aligned} \text{agregar} &= \text{toggle} \cdot \overline{\text{hit}}, \\ \text{we}_k &= \text{agregar} \cdot w_k, \\ \text{borrar}_k &= \text{toggle} \cdot \text{hit}_k. \end{aligned} \tag{6}$$

2.2.2.4.4 Bit de válido

La Tabla 6 muestra el siguiente valor de v_k. Las demás combinaciones no ocurren: $\text{we}_k = 1$ exige $v_k = 0$, $\text{borrar}_k = 1$ exige $v_k = 1$, y ambos nunca valen 1 a la vez.

v_k	we_k	borrar_k	v_k^+
0	0	0	0
0	1	0	1
1	0	0	1
1	0	1	0

Cuadro 6: Siguiente estado del bit de válido.

$$v_k^+ = (v_k + \text{we}_k) \cdot \overline{\text{borrar}_k}. \tag{7}$$

2.2.2.4.5 Contenido de la ranura

El registro de cada ranura carga `code` solo cuando $\text{we}_k = 1$, y en otro caso conserva su valor (Tabla 7).

we_k	slot_k^+
0	slot_k
1	code

Cuadro 7: Carga de la ranura (multiplexor 2 a 1).

$$\text{slot}_k^+ = \overline{\text{we}_k} \cdot \text{slot}_k + \text{we}_k \cdot \text{code}. \tag{8}$$

2.2.2.5 pwm

El módulo genera una señal PWM de N bits. La velocidad `vel` sale de la ALU con `src1`, `src2` y `op_code`. Un contador avanza solo cuando `step` vale 1, y la salida está en 1 mientras el contador sea menor que `vel`. Las tablas se muestran para $N = 3$.

2.2.2.5.1 Detección del valor máximo del contador

El contador cuenta de 0 a $2^N - 2$ (para $N = 3$: de 0 a 6, o sea 7 pasos). La Tabla 8 muestra cuándo se reinicia. El valor 7 nunca ocurre.

cnt_2	cnt_1	cnt_0	at_max
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Cuadro 8: Detección de cnt $= 2^N - 2$ ($N = 3$).

$$\text{at_max} = \overline{\text{cnt}_0} \cdot \prod_{i=1}^{N-1} \text{cnt}_i. \tag{9}$$

2.2.2.5.2 Siguiente valor del contador

Dos multiplexores encadenados deciden el siguiente valor (Tabla 9): sin `step` se conserva, y con `step` suma 1 o vuelve a 0 si llegó al máximo.

step	at_max	cnt^+
0	X	cnt
1	0	cnt + 1
1	1	0

Cuadro 9: Siguiente valor del contador.

$$\begin{aligned} \text{cnt_wrap} &= \overline{\text{at_max}} \cdot (\text{cnt} + 1), \\ \text{cnt}^+ &= \overline{\text{step}} \cdot \text{cnt} + \text{step} \cdot \text{cnt_wrap}. \end{aligned} \tag{10}$$

2.2.2.5.3 Comparador

Se calcula cnt − vel con un sumador, usando el complemento a dos: cnt + $\overline{\text{vel}}$ + 1. El acarreo de salida vale 1 cuando cnt ≥ vel, y la salida del PWM es su negado (Tabla 10).

cnt \ vel	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	1
2	0	0	0	1	1	1	1	1
3	0	0	0	0	1	1	1	1
4	0	0	0	0	0	1	1	1
5	0	0	0	0	0	0	1	1
6	0	0	0	0	0	0	0	1

Cuadro 10: `pwm_out` según el contador y la velocidad ($N = 3$).

$$\begin{aligned} \text{c_out} = 1 &\iff \text{cnt} \geq \text{vel}, \\ \text{pwm_out} = \overline{\text{c_out}} &\iff \text{cnt} < \text{vel}. \end{aligned} \tag{11}$$

2.2.2.5.4 Ciclo útil y frecuencia

Cada columna de la Tabla 10 tiene `vel` unos entre 7 filas, por lo que el ciclo útil es vel/7 (Tabla 11). Si el contador llegara a $2^N - 1$ (8 pasos), `vel` = 7 daría 87,5 % y no 100 %; por eso se cuenta solo hasta $2^N - 2$.

vel	0	1	2	3	4	5	6	7
Ciclo útil	0 %	14 %	29 %	43 %	57 %	71 %	86 %	100 %

Cuadro 11: Ciclo útil del PWM ($N = 3$).

$$\begin{aligned} D &= \frac{\text{vel}}{2^N - 1}, \\ f_{\text{pwm}} &= \frac{f_{\text{step}}}{2^N - 1}. \end{aligned} \tag{12}$$

2.2.2.6 tick_var

El módulo genera un pulso `step` de un ciclo cada div + 1 ciclos de reloj. A diferencia de un divisor fijo, el divisor llega por el puerto `div`.

2.2.2.6.1 Comparación con el divisor

No se compara `cnt` con `div` directamente: se calcula la diferencia con un sumador y se verifica que sea cero. La resta se hace con complemento a dos, $\text{cnt} + \overline{\text{div}} + 1$. La Tabla 12 resume el resultado.

Relación	diff	step
$\text{cnt} < \text{div}$	$\neq 0$	0
$\text{cnt} = \text{div}$	0	1

Cuadro 12: Generación del pulso `step`.

$$\begin{aligned} \text{diff} &= \text{cnt} + \overline{\text{div}} + 1 = \text{cnt} - \text{div}, \\ \text{step} &= (\text{diff} = 0) = \prod_{i=0}^{S-1} \overline{\text{diff}_i}. \end{aligned} \tag{13}$$

El XNOR contra cero equivale a un NOT por bit, y la AND de todos vale 1 solo si los S bits de `diff` son cero.

2.2.2.6.2 Siguiente valor del contador

Mientras `step` sea 0 el contador suma 1, y cuando vale 1 vuelve a 0 (Tabla 13).

step	cnt^+
0	cnt + 1
1	0

Cuadro 13: Siguiente valor del contador.

$$\text{cnt}^+ = \overline{\text{step}} \cdot (\text{cnt} + 1). \tag{14}$$

2.2.2.6.3 Ejemplo y período

La Tabla 14 muestra el comportamiento con div = 3: el contador recorre 0, 1, 2, 3 y el pulso aparece una vez cada 4 ciclos.

Ciclo	0	1	2	3	4	5
cnt	0	1	2	3	0	1
diff	-3	-2	-1	0	-3	-2
step	0	0	0	1	0	0

Cuadro 14: Ejemplo con div = 3.

$$T_{\text{step}} = (\text{div} + 1)\, T_{\text{clk}}. \tag{15}$$

En el diseño, con div = 7142 y $T_{\text{clk}} = 20\,\text{ns}$, el período es $7143 \cdot 20\,\text{ns} \approx 142{,}86\,\mu\text{s}$. Como el PWM necesita $2^3 - 1 = 7$ pasos por ciclo, $f_{\text{pwm}} = 1/(7 \cdot 142{,}86\,\mu\text{s}) \approx 1\,\text{kHz}$.

2.2.2.7 Sistema

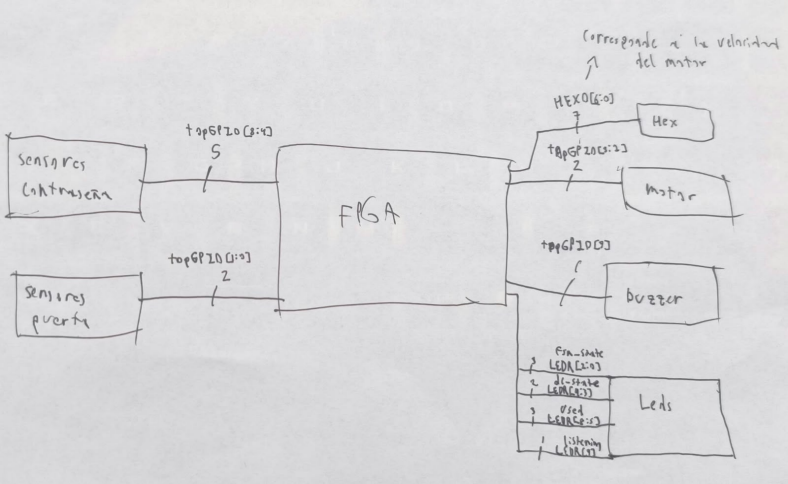


Figura 12: Diagrama de bloques que representa el comportamiento general del sistema.